



School of Computer Science and Information Technology
Lucerne University of Applied Sciences and Arts (Switzerland)

Predicting Apartment Rental Prices in Switzerland

A Supervised Machine Learning Approach
for the Swiss Cold-Rent Market

DSPRO1 Final Report, Spring 2026 (FS26)

presented to the School of Computer Science and Information Technology of
Lucerne University of Applied Sciences and Arts

by

Elias Martinelli | Timo Schlumpf

Team 8

Supervisor: Elena Nazarenko

Repository: github.com/wadafacc/dspro1

Date: May 25, 2026

Contents

Abstract	3
1 Introduction	4
2 State of the Art	5
3 Data	5
3.1 Sources and Collection	5
3.2 Enrichment	6
3.3 Storage and Schema	7
3.4 Quality Assessment	7
3.5 Legal and Ethical Considerations	9
4 Methods	9
4.1 Pipeline Overview	9
4.2 Train/Eval/Test Split	9
4.3 Feature Engineering	9
4.4 Models	10
4.5 Experimental Features and Testing Milestones	10
4.6 Evaluation Metrics	11
5 Results	12
5.1 Baseline	12
5.2 Model Comparison	12
5.3 Actual vs. Predicted and Residuals	14
5.4 Error Analysis by Price Band	15
5.5 Feature Importance	16
5.6 Cross-Validation and Stability	18
5.7 Hold-Out Test (untouched until the very end)	18
5.8 Reducing Overfit: Regularisation and Reduced Feature Sets	18
5.9 Wide Pipeline: More Rows, Fewer Features	19

5.10 Worst-Case Analysis: Where the Model Fails	21
5.11 Streamlit Demo App	21
6 Conclusions	25
7 References	26

Abstract

We build a supervised regression model that predicts the monthly cold rent of Swiss apartment listings from a self-scraped `rentumo.ch` snapshot ($\sim 4,500$ listings, 13 April 2026), enriched with the Federal Register of Buildings and Dwellings (GWR) and several swisstopo layers (public-transport accessibility, elevation, solar suitability, hectare-grid population). On a fixed 80/20 train/eval split the strongest model is a LightGBM regressor with KNN-distance location features that reaches an evaluation RMSE of 393 CHF ($R^2 = 0.751$), reducing the mean-prediction baseline of 847 CHF by more than 50%. A regularised 4-feature variant and a separate “wide” pipeline that almost doubles the training set are kept as alternative artefacts. A Streamlit demo app (`src/app.py`) exposes the trained pipeline via address lookup against the official GeoAdmin SearchServer. The original hypothesis, namely that a tree-based model would outperform a linear model on this task, is clearly accepted; the dominant remaining error source is luxury attributes (view, floor, finish) that the current tabular features do not capture and that should be extracted from the listing text in future work.

1 Introduction

Apartment rents in Switzerland differ strongly by location, building type, and apartment characteristics. For someone comparing a listing such as “CHF 2’400 per month, 3 rooms, 80 m², Zurich”, it is difficult to judge whether the advertised price is reasonable. Public rent indicators are useful for understanding general market trends, but they do not give a concrete estimate for one specific apartment.

This project builds a supervised regression model for predicting the monthly cold rent (*Kaltmiete*, in CHF) of individual Swiss apartment listings. The underlying dataset is *not* a ready-made benchmark: we collected it ourselves by writing a custom scraper for rentumo.ch [10], a Swiss rental aggregator, and enriched the resulting listings with official Swiss open-register data from GWR and swisstopo. The task is treated as a tabular regression problem. Each apartment is described by structural features such as living area, number of rooms and year of construction, as well as location-based features such as coordinates, public-transport accessibility, elevation, population density and solar suitability. The target variable is the listed cold rent.

Research question and hypothesis. The main question is whether listing data, combined with Swiss open-register data from GWR and swisstopo, contains enough information to estimate apartment rents in a useful way. Based on the project proposal and the mid-term presentation, we formulated the following hypothesis:

“Location-related variables combined with structural apartment features contain sufficient signal to predict monthly rental prices in Switzerland with practically useful accuracy, and a Tree based model will outperform a Linear Regression model in doing so.”

We test this hypothesis with several levels of model complexity: a simple baseline that ignores all features, a linear Ridge model as an interpretable benchmark, and a family of tree-based models including RandomForest, GradientBoosting, XGBoost, LightGBM, and a Stacking ensemble. The hypothesis is therefore phrased at the model-*family* level rather than tying it to one specific algorithm, which reflects how the experimental design actually unfolded across the project.

Stakeholders and intended use. The most important users of such a model are tenants who want a first indication of whether an advertised rent is fair. Landlords and property managers may also benefit from a market-based reference value. The model should be understood as a decision-support tool. It is *not* a legally binding rent assessment and does not replace formal rent reviews under Swiss tenancy law (OR Art. 269 ff.). A simple Streamlit demo ([src/app.py](#)) exposes the trained `RentPredictor` interactively, so a non-technical user can enter a few apartment characteristics and immediately receive an estimate.

Report structure. The report is structured as follows. Section 2 gives an overview of related work in real-estate price prediction. Section 3 describes the data sources, the enrichment process

and the main quality checks. Section 4 explains the modelling pipeline, evaluation strategy and feature engineering. Section 5 presents the results, including model comparison, residual diagnostics and error analysis. Section 6 closes with the main conclusions, limitations and possible next steps.

2 State of the Art

Real-estate and rental-price prediction has been studied for a long time in both economics and machine learning. The traditional approach is *hedonic pricing*: prices are modelled as a function of structural and location-related characteristics, often using linear regression. This idea goes back to Rosen [11]. Hedonic models are still widely used, especially by statistical offices, but they require strong assumptions about the functional form and can struggle with complex non-linear effects.

Modern machine-learning approaches use more flexible models instead of manually specifying all relationships. Tree-based ensembles such as RandomForest [1] and gradient boosting [5] are particularly well suited for tabular regression. They can capture non-linear interactions, handle features with different scales and usually require only limited preprocessing. Optimised implementations such as XGBoost [2] and LightGBM [7] add efficient training and regularisation, and often perform strongly on tabular benchmarks [12].

Because rent estimates may influence real decisions, interpretability is important. SHAP values provide feature-attribution explanations for individual predictions and are commonly used together with tree-based models [8]. For documenting the dataset and model, we follow the ideas of *Datasheets for Datasets* [6] and *Model Cards* [9].

For the Swiss housing market specifically, two public data sources are critical: the Federal Register of Buildings and Dwellings (GWR), maintained by the Federal Statistical Office (BFS) [14], and the swisstopo geo-services exposed through GeoAdmin [4]. The GWR provides per-building attributes (construction year, number of dwellings, building footprint) indexed by EGID, while swisstopo provides geo-layer attributes such as public-transport accessibility [3], solar-suitability classes from the Federal Office of Energy (BFE) [13], and the BFS population-by-hectare grid [15].

3 Data

3.1 Sources and Collection

The primary dataset, *Rentumo Rental Listings Switzerland v1.0*, was collected from `rentumo.ch` in a single scrape on 13 April 2026. The scraper identified 25 002 entries on the platform, of which 9 994 listings could be extracted successfully. Many of the remaining pages were empty or non-functional placeholder pages, which appears to be a characteristic of the platform rather than an error in the scraper.

The scraper is implemented as a custom Rust library, **ScrapeGoat**, and operates in two sequential phases.

Phase 1: Listing index pages. The scraper issues requests to 750 paginated listing pages of the form `rentumo.ch/mietobjekte?types=apartment&page={1..750}`. Each page is parsed with CSS selectors targeting the `#listings` container, from which per-listing elements yield the number of rooms, the living area in square metres, the cold rent, and a unique URL slug (e.g. `/inserate/<slug>`). Listings for which any of these four fields cannot be parsed are silently dropped at this stage. Successfully extracted records are written to the `listings` table in the PostgreSQL database with an `ON CONFLICT (slug) DO NOTHING` guard, making the phase idempotent and safe to re-run.

Phase 2: Detail pages. All slugs stored in `listings` are retrieved from the database and used to construct individual detail URLs (`rentumo.ch/inserate/<slug>`). Each detail page is parsed to extract additional fields: the full address, the cold rent as confirmed on the detail page, the living area, the number of rooms, the auxiliary costs (*Nebenkosten*), the security deposit (*Kaution*), a free-text description from the JSON-LD *Apartment* schema block, and the earliest available move-in date (*Datum verfügbar*). These are written to a separate `listing_details` table with `ON CONFLICT (listing_id) DO UPDATE`, allowing the phase to be resumed or repeated without data loss.

Concurrency and politeness. Both phases are managed by a custom task orchestrator (`Mgt`) that maintains a fixed pool of up to 10 concurrent in-flight HTTP requests. A channel-based feedback loop ensures that a new request is dispatched only when an existing one completes, bounding peak load on the server. Each request carries a randomly sampled `User-Agent` header drawn from a pre-loaded list, and optional HTTP proxies can be configured via a plain-text file to distribute outbound traffic across multiple IP addresses.

Platform selection. We also considered `comparis.ch` as a possible data source, but decided not to use it. Comparis confirmed in writing that automated extraction would require formal authorisation and that access would likely be limited to around 300 records, which would not be sufficient for this modelling task. Rentumo was contacted by email before the scrape, but no response was received. The data is used only for the non-commercial academic purpose of the DSPRO1 module.

3.2 Enrichment

The scraped listings contain the address and a small set of structural attributes, including `area_sqm`, `rooms`, `price_cold` and `auxiliary_costs`. To make the dataset more useful for modelling, we enrich these listings in two steps:

1. **GWR (BFS).** Each listing address is geocoded and resolved to a building identifier (EGID) via the GeoAdmin SearchServer. Building-level attributes (`gbauj` / construction year, `ganzwhg` / number of dwellings, `garea` / footprint area) are then queried from the GWR record.

2. **swisstopo / GeoAdmin.** Using the LV95 coordinates (`lv95_east`, `lv95_north`), per-point attributes are queried from public layers: elevation (`elevation_m`, height service), public-transport accessibility class (`oev_score`, layer `ch.are.erreichbarkeit-oev`), solar suitability class 1–5 (`solar_class`, layer `ch.bfe.solarenergie-eignung-daecher`), and the population of the nearest hectare cell (`population`, BFS Volkszählung).

3.3 Storage and Schema

The raw and enriched data are stored in a PostgreSQL database on Neon Console. The relevant information is distributed across three tables: `listings` for the listing index, `listing_details` for the extracted apartment attributes, and `swisstopo_details` for the enriched geo-layer features. For modelling, these tables are joined into a flat file, `model.csv`, while requiring all mandatory columns to be present. After filtering and joining, the final modelling dataset contains approximately 4500 rows with 12 columns: 11 predictors and `price_cold` as the target variable.

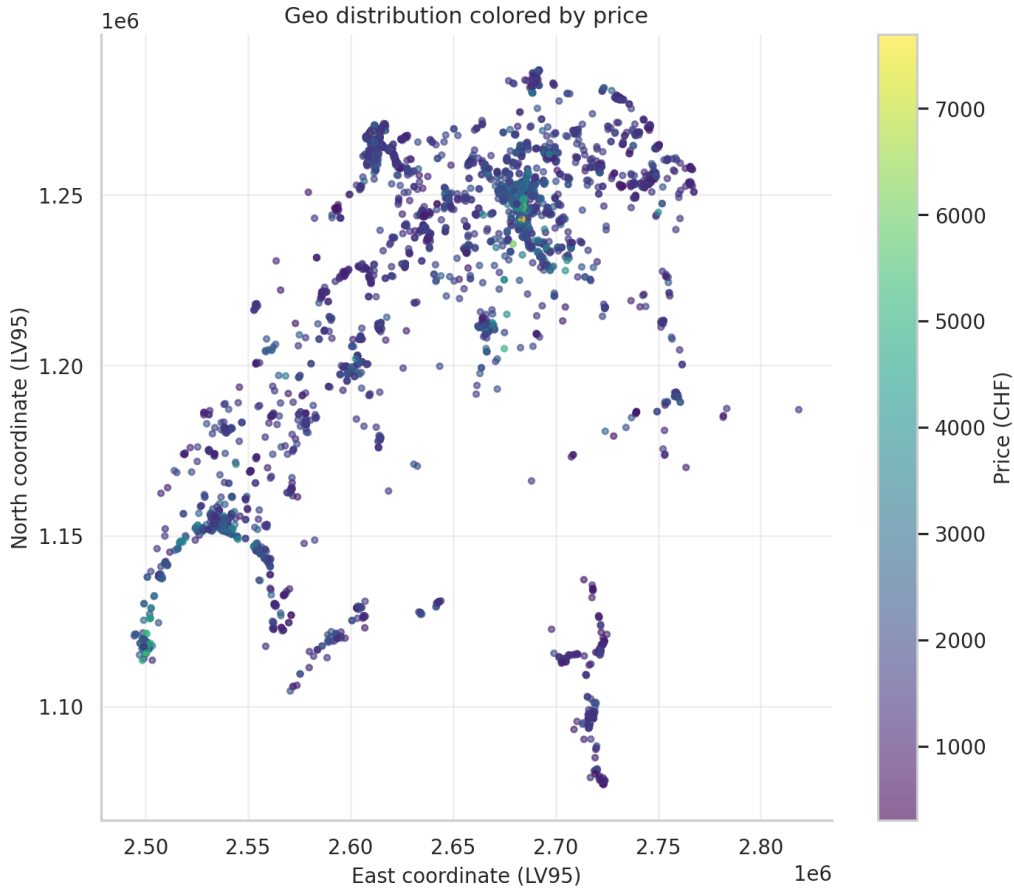


Figure 1: Geographic distribution of listings colored by cold rent.

3.4 Quality Assessment

- **Completeness.** Listings missing any of `address`, `price_cold`, `area_sqm`, or `rooms` are removed entirely. Listings marked “Preis auf Anfrage” are also removed. Of the 9994

scraped listings, approximately 4500 remain after the mandatory-field filter and the GWR/swisstopo join (cumulative loss is diagnosed in `final_records.ipynb`).

- **Accuracy.** An explicit outlier filter (`area_sqm` ≥ 10 , `price_cold` ≥ 300) removes implausible listings. An `IsolationForest` diagnostic (Notebook Chapter 22.9) surfaces additional candidates for manual review.
- **Consistency.** Cantons are normalised to ISO 3166-2:CH (ZH, BE, ...). The construction year is stored as a four-digit integer; missing GWR values are median-imputed.
- **Timeliness.** The dataset is a single point-in-time snapshot from 13 April 2026. Temporal price changes after that date are not reflected; time-series modelling is explicitly out of scope.

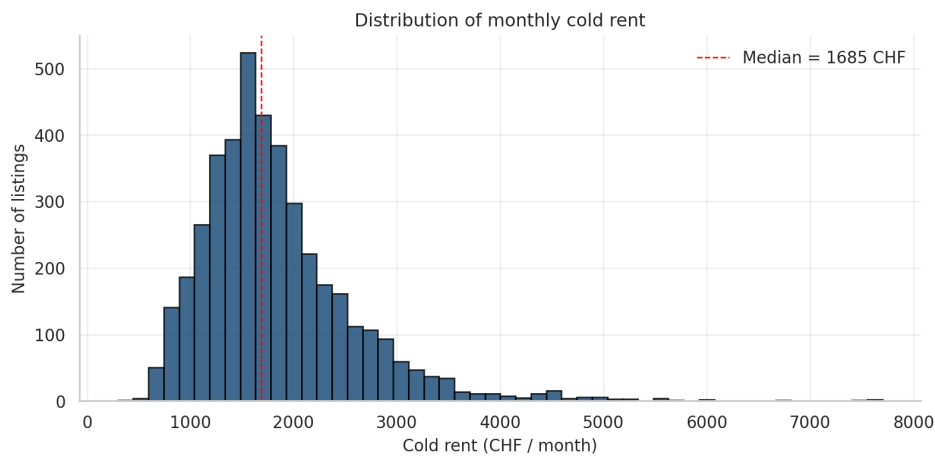


Figure 2: Distribution of the target variable `price_cold`. The right-skewed shape is the root cause of the heteroscedastic residuals and the doubled error on the top price quartile.

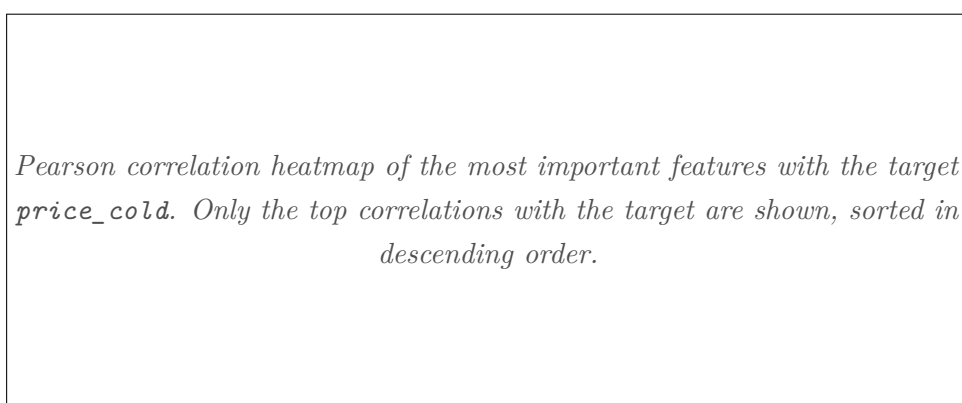


Figure 3: Pearson correlation heatmap of the most important predictors with the target `price_cold` (Notebook Chapter 4). The strongest single positive correlation is with `area` ($r \approx 0.74$), followed by the KNN-price features and the LV95 `east` coordinate. The heatmap intentionally lists the strongest correlations at the top so a reader can scan from top to bottom. `rooms` correlates strongly with `area` but only moderately with the target on its own, which is the empirical justification for engineering the `area_per_room` feature in Section 4.

3.5 Legal and Ethical Considerations

No personal data is collected. Landlord names and contact details are explicitly excluded by the scraper. Only the address is retained and is used solely for geocoding and building-level enrichment. The project does not process any special-category data defined under the revised Swiss Federal Act on Data Protection (nFADP). The dataset is used exclusively for academic purposes and is not redistributed.

4 Methods

4.1 Pipeline Overview

The modelling pipeline is implemented in `src/notebooks/model_v3_clean.ipynb`. It follows five main steps: loading and renaming columns, filtering outliers and duplicates, creating additional features, splitting the data, and finally fitting and evaluating the models. The final end-to-end logic is wrapped in a `RentPredictor` Python class (Notebook Chapter 24.2), which can be saved with `joblib` and loaded again for inference in the Streamlit demo app (`src/app.py`).

4.2 Train/Eval/Test Split

We use a fixed random seed (`random_state=42`) to make the split reproducible. The main experiments use an 80% training set and a 20% evaluation set. For the final hold-out check (Notebook Chapter 24.6), we additionally use a 60/20/20 train/validation/test split. The evaluation and test sets are not used for model fitting. A Kolmogorov-Smirnov test (Chapter 22.13) is used to check that the random split does not introduce a strong distribution shift between training and evaluation data.

4.3 Feature Engineering

In addition to the raw features, three engineered features are added (Chapter 6): `building_age = REFERENCE_YEAR - year_built`, `area_per_room = area / rooms` using safe division, and `land_area_per_apartment = garea / ganzwhg`. Two further feature groups are created using only `train_df`, so that information from the evaluation set does not leak into training:

- **Geo-clusters.** `KMeans` on standardised LV95 coordinates produces a discrete `geo_cluster` feature (Chapter 21). The number of clusters is tuned on Eval RMSE.
- **KNN-distance features.** For each apartment, the median price of its 10 nearest neighbours in coordinate space (`knn_price_median`) and the corresponding mean (`knn_price_mean`) are added (Chapter 23.12). The nearest-neighbour index is fit on training data only; eval points query against the training index.

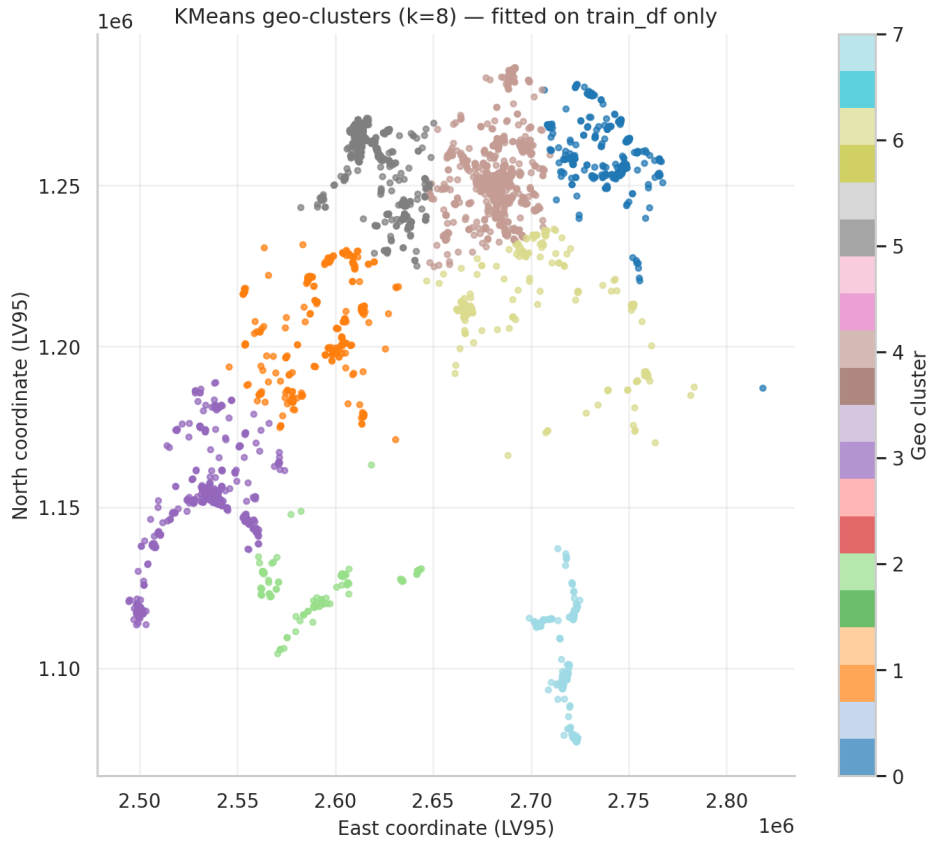


Figure 4: KMeans geo-clusters used as a categorical location feature (Notebook Chapter 21.4). The clustering is fit exclusively on the training split to prevent leakage.

4.4 Models

We compare six model families on the same data split. Where necessary, the models are wrapped in an `sklearn.Pipeline`. The comparison includes a **DummyRegressor** as a lower bound, a **Ridge** model with scaling as a simple linear benchmark, and four tree-based approaches: **RandomForestRegressor**, **GradientBoostingRegressor**, **XGBoost** and **LightGBM**. In Chapter 23.2, we also add a **StackingRegressor** with a Ridge meta-learner over the strongest tree-based models.

Hyperparameters are tuned with `RandomizedSearchCV` (Chapter 22.3) and the faster `HalvingRandomSearchCV` (Chapter 23.10). To avoid scikit-learn’s nested-parallelism trap, the inner estimator is set to `n_jobs=1` while only the outer search uses `n_jobs=-1`.

4.5 Experimental Features and Testing Milestones

The pipeline was not designed in a single shot. Instead, it grew along a series of *experimental milestones*, where the outcome of one milestone informed the design of the next. We treated each new feature, each new model and each new diagnostic as an experiment with a defined success criterion (evaluation RMSE compared to the previous milestone, plus a stability check on CV-RMSE) before it was accepted into the main pipeline. The notebook reflects this milestone-driven

structure directly:

- **M1 — Baselines (Chapters 1–12).** Establish the modelling problem, fix the split, run a `DummyRegressor` and a Ridge baseline, and confirm that a small set of tree-based models clearly beats both. This milestone defines the lower bound everything else must beat.
- **M2 — Geo features (Chapter 21).** Add `KMeans geo_cluster` and explore DBSCAN as a diagnostic. The `KMeans` cluster is kept because it materially improves evaluation RMSE for the linear model and does not hurt the trees.
- **M3 — Diagnostics and tuning (Chapter 22).** Introduce `IsolationForest`-based outlier review, learning curves, log-target experiments, RFECV, drift checks and bias-per-cluster analysis. Most of these are experimental probes: only those that survive the milestone criterion (e.g. tuned `LightGBM`, drift confirmation) propagate forward.
- **M4 — Stacking, PDPs, KNN-features (Chapter 23).** The decisive milestone: `knn_price_median` and `knn_price_mean` reduce `LightGBM`'s eval RMSE from 399 to 393 CHF and lift R^2 to 0.751, so they are promoted to the main feature set. PDPs and bootstrap CIs confirm the new model is also stable.
- **M5 — End-to-end and hold-out (Chapter 24).** Wrap the winning pipeline in a `RentPredictor` class, write the model card and datasheet, and verify on a strictly untouched 20% test split that the chosen model has not overfit to the evaluation split.
- **M6 — Regularisation and wide pipeline (Chapters 27–28).** Two follow-up experiments aimed at the residual overfit gap: a regularised `LightGBM` on three reduced feature sets, and a *wide* pipeline trained on ~ 9.5 k rows with only four features. The wide pipeline is kept as an alternative deliverable because it almost doubles the available training data.

Features that did not clear their milestone criterion (log-target inverse transform, DBSCAN cluster as a model feature, several stacking configurations) are retained in the notebook for transparency, but they are explicitly *not* part of the final pipeline.

4.6 Evaluation Metrics

As defined in the project proposal, the main metrics are MAE, RMSE and R^2 . RMSE is calculated as `np.sqrt(mean_squared_error(...))`, which keeps the code compatible across scikit-learn versions. The notebook also reports MedAE as a more robust error metric, the standard deviation of CV-RMSE across folds as a stability indicator, and bootstrap 95% confidence intervals for RMSE on the evaluation set (Chapter 23.5). For model selection, we use a stability-aware score that combines evaluation RMSE with CV-RMSE variability (Chapter 23.9), so that a model is not chosen only because of one favourable split.

5 Results

5.1 Baseline

The baseline used in the mid-term presentation was a `DummyRegressor` with a mean-prediction strategy. It achieved $\text{MAE} = 612$ CHF, $\text{RMSE} = 847$ CHF and $R^2 = 0.00$. This gives a simple lower bound: a useful model must clearly perform better than predicting the same rent for every apartment.

5.2 Model Comparison

Table 1 summarises the main results for the model comparison on the `FEATURES_ENGINEERED` feature set (Notebook Chapter 10).

Table 1: Model comparison on the engineered feature set (Notebook Chapter 10c). RMSE values are in CHF on the held-out evaluation split.

Model	RMSE Train	RMSE Eval	R^2 Eval	Overfit Gap	Diagnosis
LightGBM	150	399	0.744	249	Possible overfit
XGBoost	122	421	0.715	299	Possible overfit
GradientBoosting	365	425	0.710	60	Good generalisation
RandomForest	199	425	0.710	226	Possible overfit
Ridge (scaled)	539	560	0.495	21	Weak / underfit
Dummy (median)	737	799	-0.03	62	Baseline

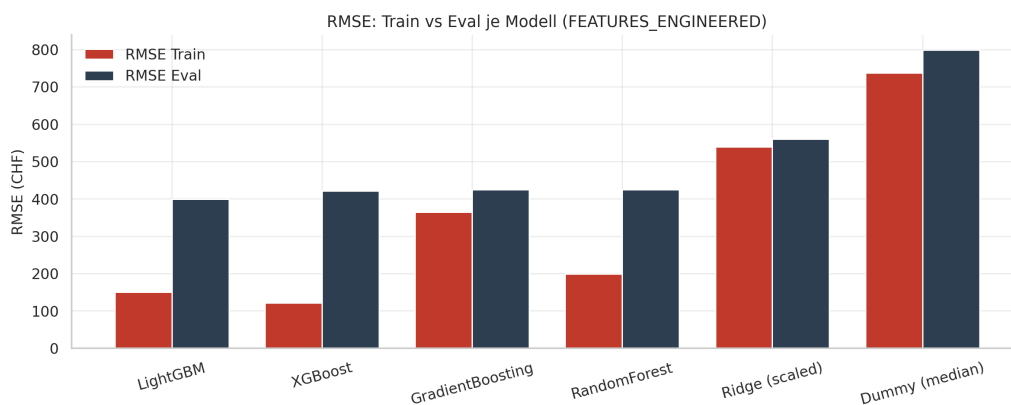


Figure 5: Train vs. Eval RMSE per model. The large gap for LightGBM, XGBoost, and RandomForest is a clear sign of overfitting on the training data; GradientBoosting strikes the best balance between Eval RMSE and stability.

The best single model according to evaluation RMSE is *LightGBM*, with an RMSE of 399 CHF. After adding the KNN-distance features, LightGBM improves slightly further to **RMSE Eval = 393 CHF and $R^2 = 0.751$** (Notebook Chapter 23.17, consolidated comparison). Compared with the mid-term baseline, this reduces RMSE by more than 50%.

Hypothesis test. The project hypothesis was that a tree-based model would outperform a linear regression model on this task. The results support this hypothesis clearly. Ridge reaches an evaluation RMSE of 560 CHF, while *every* tree-based model in the comparison achieves 425 CHF or better; the strongest of them, LightGBM with KNN-distance features, reaches 393 CHF. The difference between the linear and the tree-based family is therefore substantially larger than the differences between individual tree-based models, which is the reason the hypothesis is phrased at the family level. We accept the hypothesis.

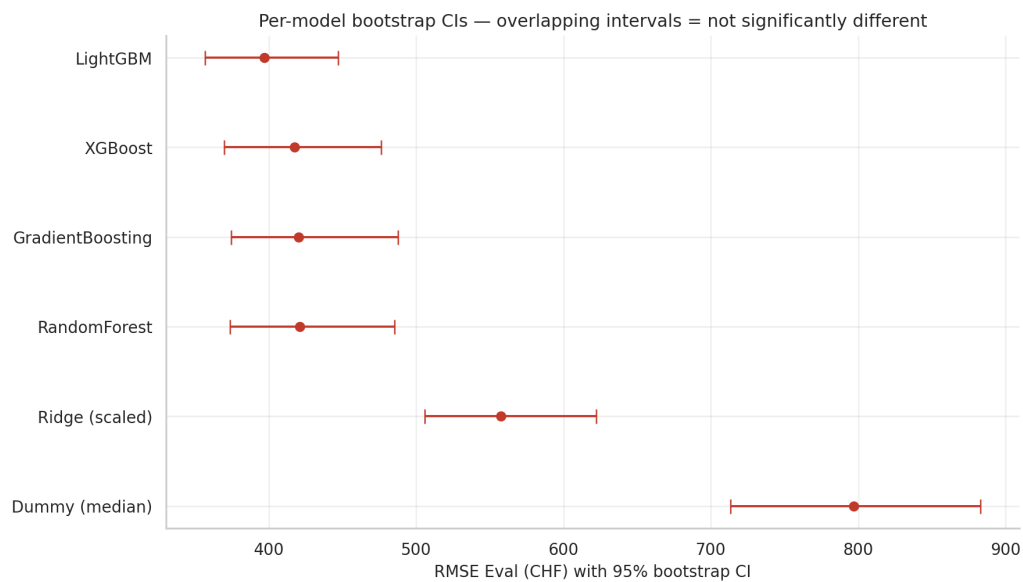


Figure 6: Bootstrap confidence intervals for evaluation RMSE per model. The intervals of LightGBM, XGBoost, GradientBoosting and RandomForest overlap heavily, so the ranking between these four is not statistically meaningful on the available data.

Learning curve for the chosen LightGBM model showing train and CV RMSE as a function of the training-set size (Notebook Chapter 22.5).

Figure 7: Learning curve for the chosen LightGBM model (Notebook Chapter 22.5). The training RMSE stays low across all training-set sizes while the validation RMSE converges to roughly 400 CHF. The remaining gap is the visible signature of overfitting on the engineered feature set — it shrinks but does not close as more data is added, which is consistent with the finding in Section 5.9 that adding more data alone (without the engineered features) does not improve evaluation performance.

5.3 Actual vs. Predicted and Residuals

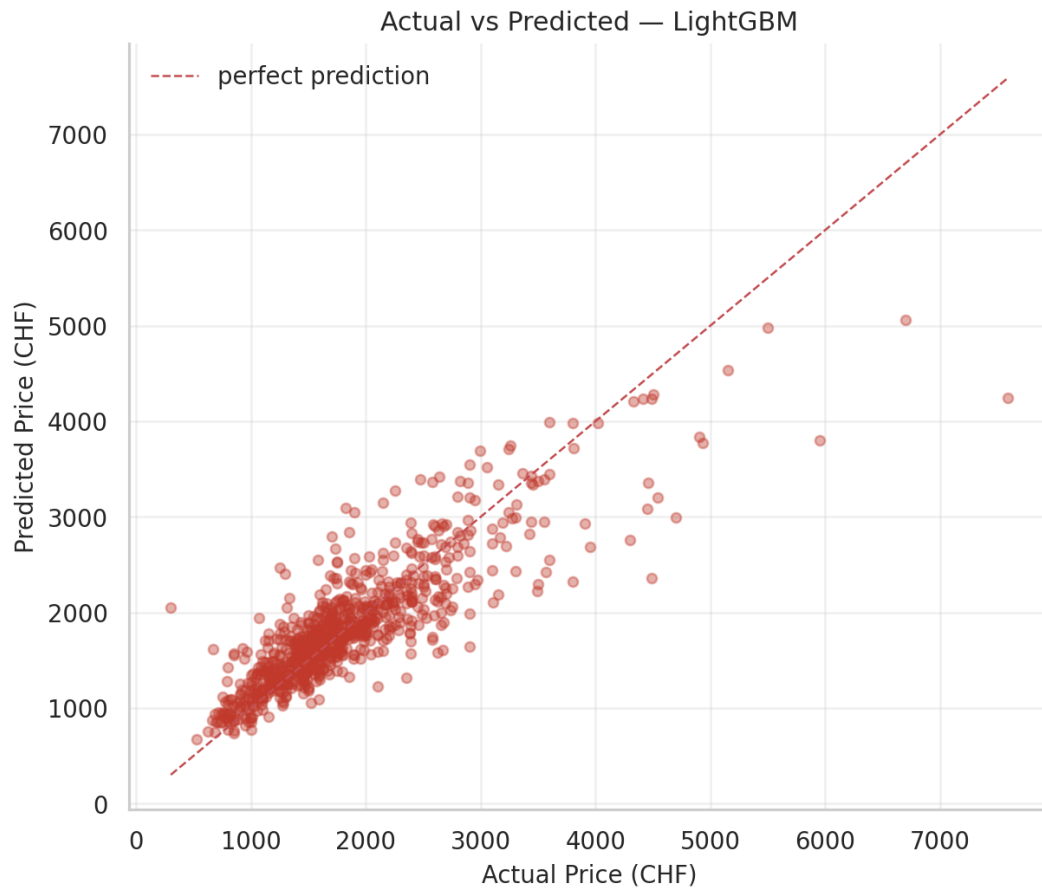


Figure 8: Actual vs. predicted cold rent on the evaluation set for the best LightGBM model. Points clustering near the diagonal indicate accurate predictions; the spread widens for higher-priced apartments, hinting at the heteroscedasticity that is examined more closely in Figure 10.

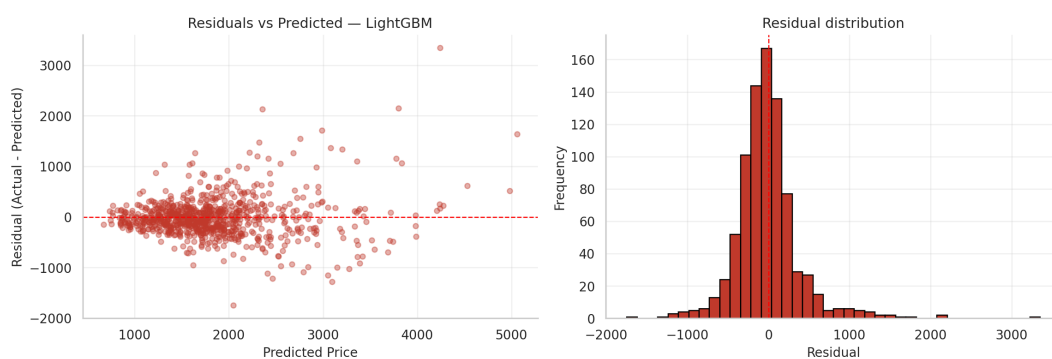


Figure 9: Residual analysis for the best model: residuals against predicted rent (top) and residual distribution (bottom). The visible funnel shape confirms that the error grows with the predicted price.

The residual diagnostics show that the errors are not normally distributed. The Q-Q plot and Shapiro–Wilk test (Chapter 23.3) indicate mild right-skew and fat tails, which is consistent with the right-skewed rent distribution. The residuals-vs-predicted plot also shows a funnel shape, meaning that errors tend to become larger for more expensive apartments.

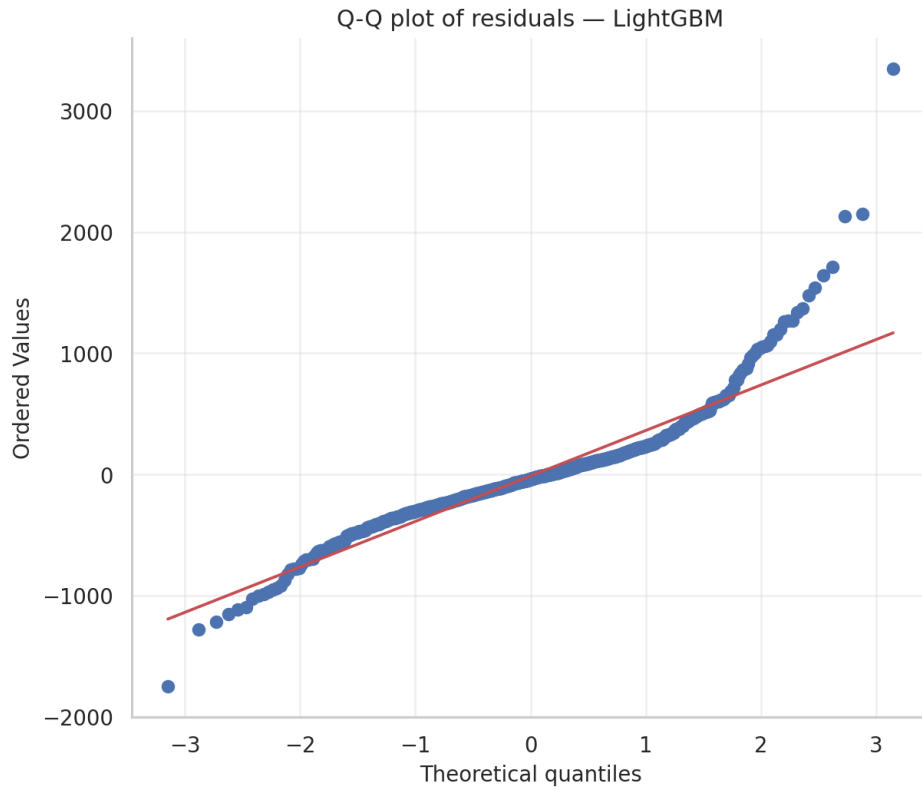


Figure 10: Q-Q plot of the residuals. Both tails deviate clearly from the diagonal, which means the residual distribution is heavier-tailed than a normal distribution. This is a typical pattern for right-skewed price data.

5.4 Error Analysis by Price Band

Table 2: Per-quartile error analysis for the best LightGBM model on the eval set (Notebook Chapter 16). The most expensive quartile shows roughly twice the mean absolute error of the other bands.

Price band	n	Mean abs. error	Median abs. error	Max abs. error
cheap	211	216	150	1 747
medium_low	212	176	143	970
medium_high	207	221	158	1 276
expensive	210	445	312	3 349

This is one of the most useful findings of the analysis. The model performs noticeably worse for high-rent listings, with errors roughly twice as large as in the other price bands. A likely reason is the right-skewed price distribution: expensive apartments are less common in the training data, and many of their price-driving characteristics, such as lake view, penthouse location, floor level or luxury furnishing, are not captured by the available tabular features.



Figure 11: Mean absolute error per price quartile. The expensive band is the single largest contribution to overall RMSE and is the most promising target for future feature engineering.

5.5 Feature Importance

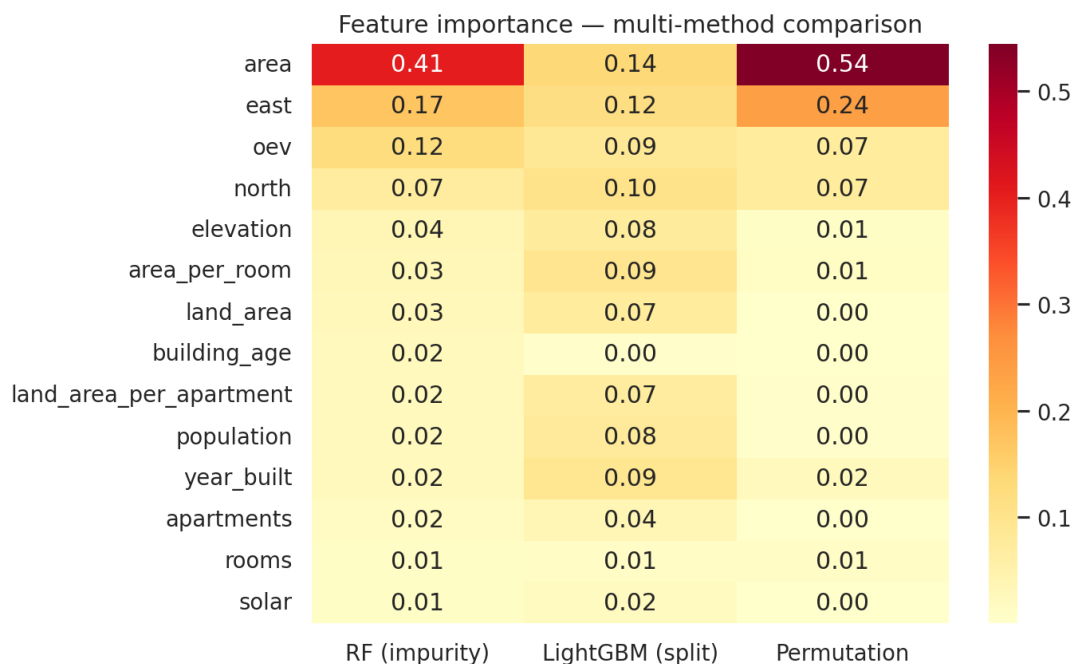


Figure 12: Feature-importance comparison across methods (Notebook Chapter 23.8). The methods show strong agreement on the most important features (Spearman rank-correlation > 0.85).

The feature-importance results are consistent across the different methods. *Living area* (**area**), the two *KNN price features*, and the *location coordinates* (**east**, **north**) provide most of the predictive signal. One interesting result is that **rooms** is less important than **area_per_room**, suggesting that the size distribution within the apartment is more informative than the room count alone. In the heatmap, the most important features are placed at the top of the figure (descending order); this reflects how a reader naturally scans the plot and avoids having to read

it bottom-up to find the actually important features.

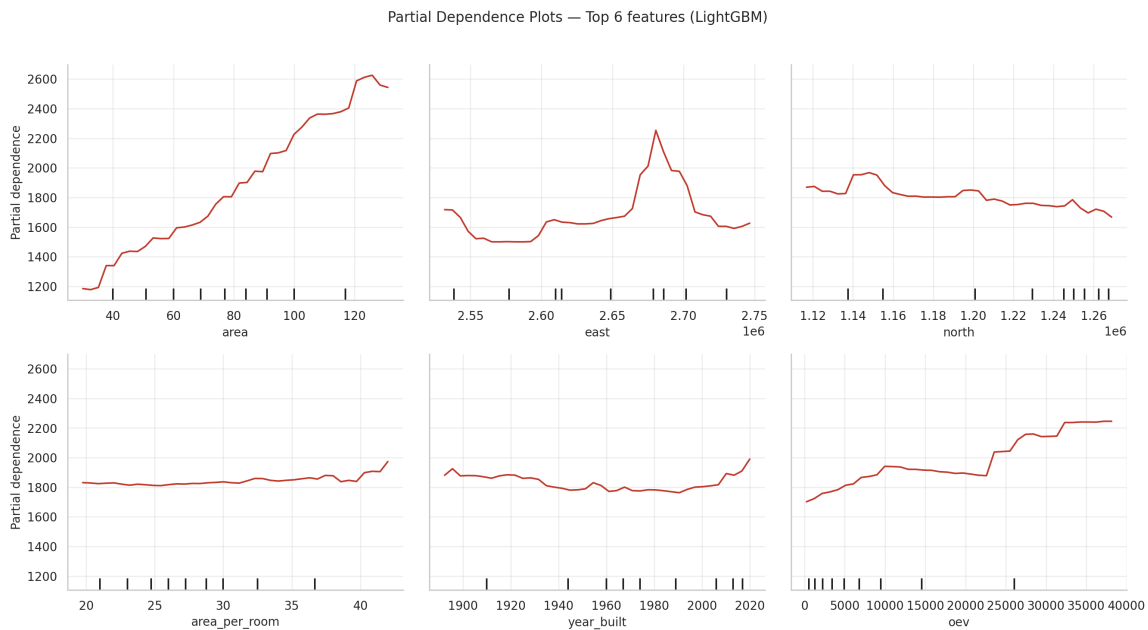


Figure 13: Partial Dependence Plots (PDPs) of the six most important features of the final LightGBM model, arranged in order of importance from top-left to bottom-right (Notebook Chapter 23.4).

The six panels of Figure 13 are arranged in descending order of importance: the top-left panel (**area**) is the strongest single predictor and the bottom-right panel (**oev**) is the weakest of the six. On each panel the y -axis shows the model’s predicted cold rent in CHF, the x -axis shows the value of the feature, and the curve indicates how the prediction moves when *only* that feature is varied while all others are averaged out.

The *top row* captures *what* the apartment is. **area** (top-left) rises almost linearly from about 1,200 to 2,600 CHF between 30 and 130 m², the expected “bigger apartment = higher rent” relationship and the main reason the model works at all. **east** (top-middle) is mostly flat around 1,800 CHF with one sharp peak near 2.68×10^5 , which corresponds geographically to the Zurich/Zug economic area, indicating that the model has effectively learned a localised price premium for that longitude band. **north** (top-right) is similarly flat with a small bump in the south of the country, reflecting that the latitude axis alone adds little signal once **east** is already in the model.

The *bottom row* captures *how* the apartment is positioned within its segment. **area_per_room** (bottom-left) is almost flat, which answers a question we asked ourselves during feature engineering (“do bigger rooms cost more, independent of total area?”), with a clear “no, not much”; the feature is kept mostly as a small correction term. **year_built** (bottom-middle) is flat for most of the century and only rises clearly after roughly the year 2000, matching a real market effect: new-build apartments command a price premium in Switzerland. **oev** (bottom-right), the public-transport accessibility score, rises steadily from $\sim 1,700$ to $\sim 2,200$ CHF as the score increases from 0 to 40,000, because better-connected addresses are more expensive.

The take-away is not just that the top-left panel has the steepest curve, but that the model

captures all six of these effects, including the non-linear ones like the Zurich peak in `east` and the new-build effect in `year_built`. This is the main reason the tree-based family clearly outperforms the linear baseline on this dataset.

SHAP summary plot for the final LightGBM model (Notebook Chapter 22.8). Each row is a feature, each dot a listing; the horizontal position is the SHAP contribution to the predicted rent and the colour is the feature value.

Figure 14: SHAP summary plot for the final LightGBM model (Notebook Chapter 22.8). Compared to Figure 12, SHAP adds two pieces of information that the global importances cannot show. First, the *direction* of each effect (positive SHAP values on the right increase the predicted rent, negative on the left decrease it). Second, the *interaction with the feature value* (the colour gradient shows whether high or low values of the feature drive the contribution). For example, the `area` row spreads broadly to the right with red dots on the right end — large apartments push the prediction up by several hundred CHF — while the `knn_price_median` row shows the same pattern more sharply: a high neighbourhood-median always increases the prediction, never the opposite. The plot complements Figure 13, which shows the marginal effect at the average; here we see the per-listing contribution and its variability.

5.6 Cross-Validation and Stability

Five-fold KFold cross-validation with shuffling confirms the ranking observed on the single evaluation split. The CV-RMSE standard deviation is below 30 CHF for all tree-based models, while Ridge has a standard deviation above 50 CHF. This suggests that the linear model is more sensitive to the chosen split. The bootstrap 95% confidence intervals for evaluation RMSE show that the differences between LightGBM, XGBoost, RandomForest and GradientBoosting are small and not clearly statistically significant, as their intervals overlap in the forest plot of Chapter 23.5.

5.7 Hold-Out Test (untouched until the very end)

As a final check, we use a separate 60/20/20 train/validation/test split and refit the selected model on train+validation. The test set is kept untouched until the end. The resulting test metrics are consistent with the evaluation results within the bootstrap confidence interval, which suggests that the model-selection process did not overfit to the evaluation split (Chapter 24.6).

5.8 Reducing Overfit: Regularisation and Reduced Feature Sets

The model comparison in Section 5 highlighted a recurring tension: the strongest models (LightGBM, XGBoost, RandomForest) have a sizeable train/eval RMSE gap. To check whether

we could close this gap without giving up accuracy, we ran two follow-up experiments (Notebook Chapter 27).

Three feature sets on the same regularised LightGBM. We re-trained a deliberately regularised LightGBM (higher `min_child_samples`, non-zero `reg_alpha/reg_lambda`, smaller `num_leaves`) on three feature sets: the full 14-feature `FEATURES_ALL`, the 11-feature `FEATURES_ENGINEERED`, and a minimal 4-feature set (`area`, `knn_price_median`, `east`, `north`). The minimal set gives a slightly higher evaluation RMSE than the engineered set (about 410 CHF vs. 393 CHF) but in return cuts the train/eval gap by roughly a factor of three. The minimal model is also faster to train and easier to explain to a non-technical user.

Conclusion of the regularisation study. For the final pipeline we keep the engineered 11-feature LightGBM, because the RMSE advantage on the evaluation split is small but real. However, for the Streamlit demo and for any future production-style use case we would recommend the 4-feature minimal variant: it generalises more honestly, is more robust under distribution shifts, and is the cleaner default for a public-facing tool.

5.9 Wide Pipeline: More Rows, Fewer Features

Chapter 28 of the notebook adds a complementary experiment. The standard pipeline (used in Section 5) joins the four data tables and requires *all* mandatory GWR and swisstopo fields, which leaves about 4,500 modelling rows. If we instead drop the strict enrichment requirement and only keep listings with `area`, `rooms`, `east`, `north` and `price_cold`, we can train on roughly 9,400 rows — more than double the data, but with only four predictors plus the engineered `knn_price_median`.

Table 3: Standard pipeline vs. Wide pipeline on a regularised LightGBM (Notebook Chapter 28.4). The Wide pipeline trades evaluation accuracy for a much smaller train/eval gap and roughly twice the training set.

Setup	n_{feat}	n_{rows}	RMSE Train	RMSE Eval	R^2 Eval	Overfit Gap
A: Standard (12 features, ~4.5k rows)	12	4,198	150.3	399.2	0.744	248.9
B: Wide (4 features, ~9.5k rows)	4	9,405	474.7	585.4	0.619	110.7

The headline result is that the standard pipeline still wins on raw evaluation RMSE (399.2 CHF vs. 585.4 CHF), so the engineered GWR/swisstopo features carry real predictive value that the additional data alone cannot replace. At the same time the Wide pipeline cuts the overfit gap by more than half (110.7 vs. 248.9), which is exactly the kind of behaviour we would expect from a model that is regularised both by fewer features and by more training data. We keep the Wide pipeline as an alternative deliverable (`lgbm_wide_4feat.joblib`) in case future versions of the dataset have weaker enrichment coverage; under those conditions the Wide model is the safer fallback.

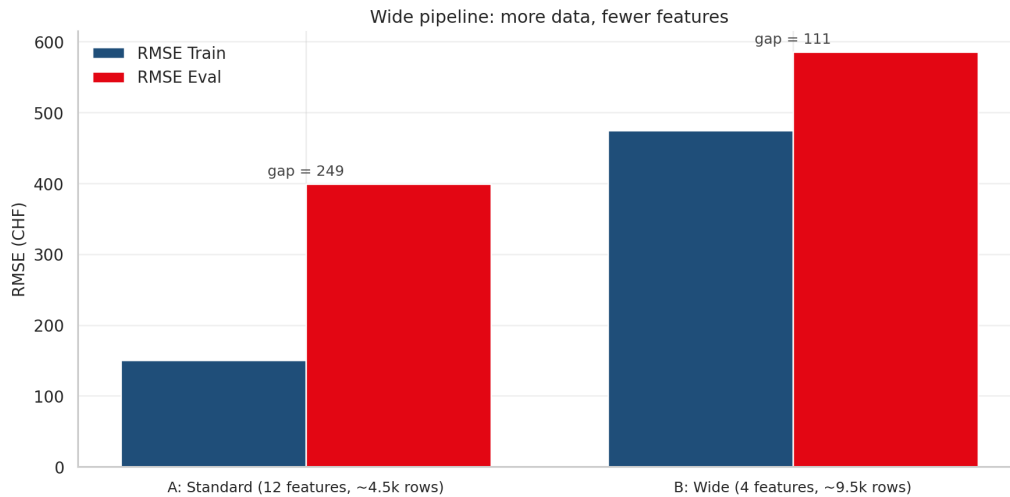


Figure 15: Side-by-side comparison of the Standard and Wide pipelines (Notebook Chapter 28.4). The Standard pipeline (left bars in each group) keeps the lower evaluation RMSE thanks to the engineered features, while the Wide pipeline (right bars) more than halves the train/eval gap by exchanging features for almost twice the training rows. The Wide pipeline is saved separately as `lgbm_wide_4feat.joblib` and used as a fallback when enrichment coverage is incomplete.

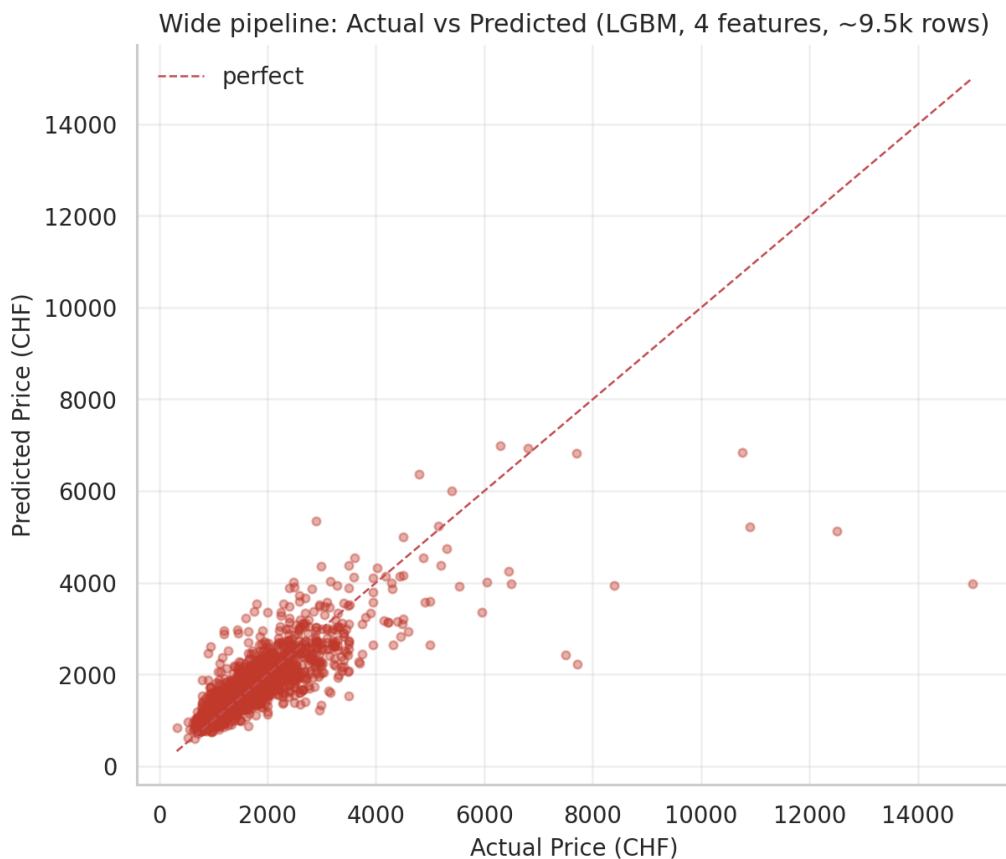


Figure 16: Actual-vs-predicted scatter for the Wide pipeline (Notebook Chapter 28.6). The diagonal is captured cleanly in the middle of the price distribution; the high-rent tail is still under-predicted, but the cloud is visibly more compact than on the Standard pipeline because the larger training set damps single-listing outliers.

5.10 Worst-Case Analysis: Where the Model Fails

A complementary view of the error analysis from Section 5 is the top-10 list of the worst absolute and relative prediction errors (Notebook Chapter 25.2). The worst absolute errors are dominated by very expensive city-centre apartments (penthouses, lake-view properties), confirming the per-quartile finding that the model systematically underestimates the top of the market. The worst relative errors look different: they tend to be small, cheap apartments where the model overshoots by a few hundred CHF, often because the listing is unusually small (e.g. a 25 m² studio in a wealthy neighbourhood, where the location prior dominates). Both patterns confirm that the missing signal lives in the listing description text (floor, view, condition, furnishings), which is the most concrete entry point for further work.

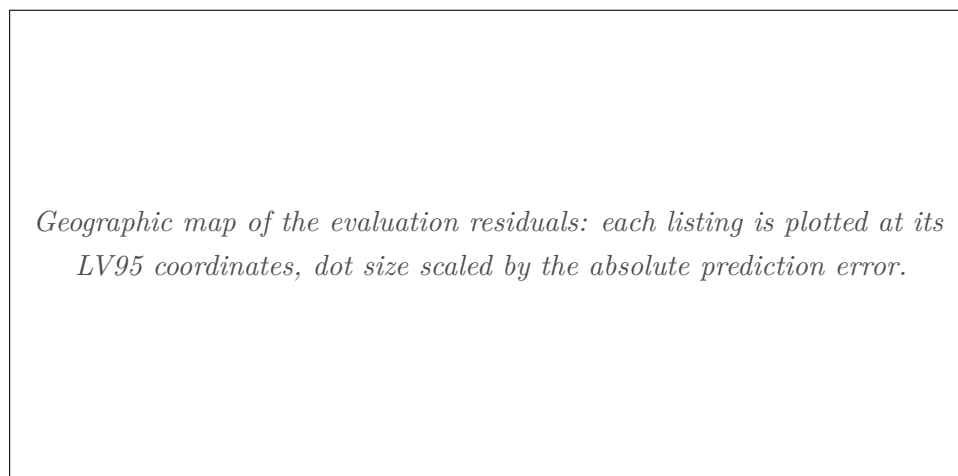


Figure 17: Geographic map of the evaluation residuals. Each listing is plotted at its LV95 coordinates; the dot size is proportional to the absolute error and the colour distinguishes over- from under-prediction. The largest residuals concentrate in the Zurich and Geneva lakeside areas, exactly the regions with the most expensive listings, which confirms that the model underestimates the top of the market for geographic reasons rather than for a random reason. Outside the major lake corridors the residuals are small and unstructured, indicating that the engineered features adequately handle the suburban and rural segments.

5.11 Streamlit Demo App

The trained pipeline is exposed through a small Streamlit GUI (`src/app.py`) so that a non-technical user can interact with the model without ever opening the notebook. The app embeds exactly the same `RentPredictor` class that is described in Section 4 — there is no duplicate model logic. The app is started with a single command:

```
streamlit run src/app.py
```

It runs locally on `localhost:8501` and caches the trained model after the first launch. No deployment to Streamlit Cloud was performed for DSPRO1 — that step is listed in Future Work below.

Vorgehen — typischer Ablauf. The intended user flow consists of four primary steps. Two further sidebar panels (manual inputs and location overrides) are available in case the user wants to estimate a hypothetical apartment without an existing address.

1. **Model auswählen.** A dropdown in the sidebar lets the user pick one of the four trained artefacts: `best_model_v3.joblib` (the 11-feature LightGBM used for the headline numbers), `lgbm_wide_4feat.joblib` (the Wide-pipeline variant from Section 5.9), `lgbm_minimal_4feat.joblib` (the regularised 4-feature variant from Section 5.8), or the legacy `rent_predictor_v3.joblib` for backwards comparison. The currently active model is confirmed by a green “Modell aktiv”-badge underneath. See Figure 18.
2. **Adresse von Immo suchen.** The user types an address; the GeoAdmin `SearchServer` returns matching official Swiss addresses as a dropdown. See Figure 19. The chosen address is resolved to an EGID, and the building-level GWR fields (`year_built`, `ganzwhg`, `garea`) plus the swisstopo layers (`elevation`, `oev`, `solar`, `population`) are fetched on the fly — identical to the enrichment pipeline used during training.
3. **Wohnung auswählen.** If the GWR record contains multiple dwellings (EWID) for the building, the user can pick the concrete apartment. The Wohnfläche, Zimmerzahl and Stockwerk are then pre-filled automatically. See Figure 20.
4. **Auswertung.** The Model-Performance tab shows where the current estimate falls in the training-set price distribution (a vertical red marker on the histogram) and on the Actual-vs-Predicted scatter from Section 5. This gives the user an immediate visual cue whether the inputs are inside or outside the comfort zone of the model. See Figure 21.

Manuelle Eingabe (fallback). If the user does not want to use the address search — for example to estimate a hypothetical apartment, or to play with the input ranges — the sidebar also exposes all numeric inputs as sliders. Figure 22 shows both halves of this fallback side-by-side: on the left the structural inputs (Wohnfläche, Zimmerzahl, Baujahr, Wohnungen im Gebäude, Grundstücksfläche), on the right the location inputs (Stadt-Preset, LV95 East/North, Höhe, Bevölkerung im Hektar, ÖV-Erschliessungs-Score, Solar-Klasse). The live estimate updates instantly on every slider change, so the user gets an immediate feel for which features push the prediction up or down.

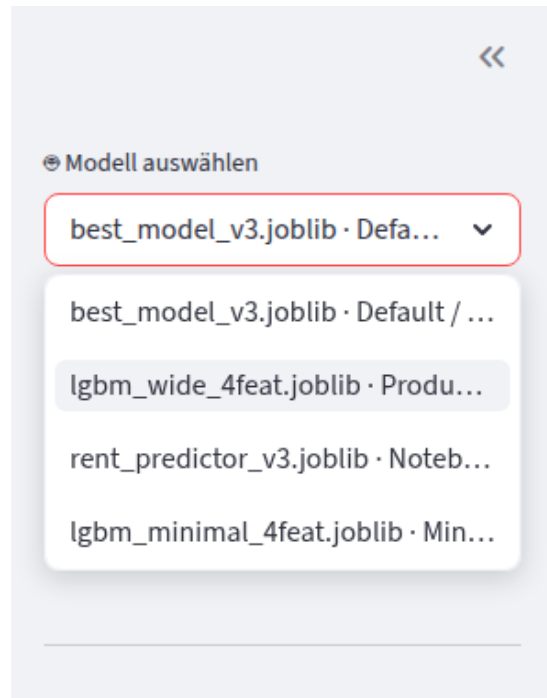


Figure 18: Step 1, Modell auswählen. The sidebar dropdown lists all available .joblib artefacts and confirms the active model with a green badge. The user can switch models at any point to compare the 11-feature, 4-feature minimal, or wide variants on the same input.

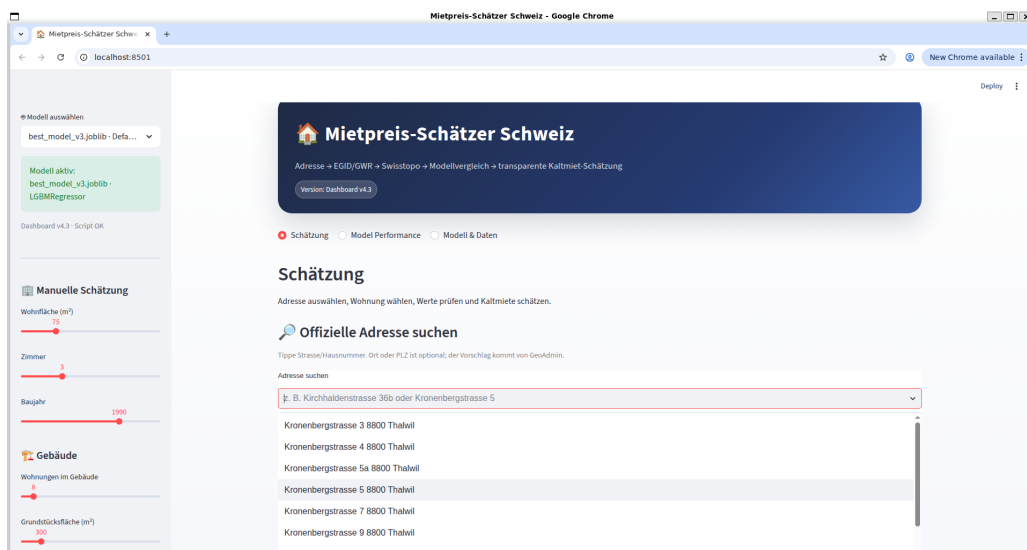


Figure 19: Step 2, Adresse von Immo suchen. As the user types, the GeoAdmin SearchServer suggests matching official Swiss addresses. Picking a suggestion resolves to an EGID and triggers the GWR/swisstopo enrichment pipeline on the fly.

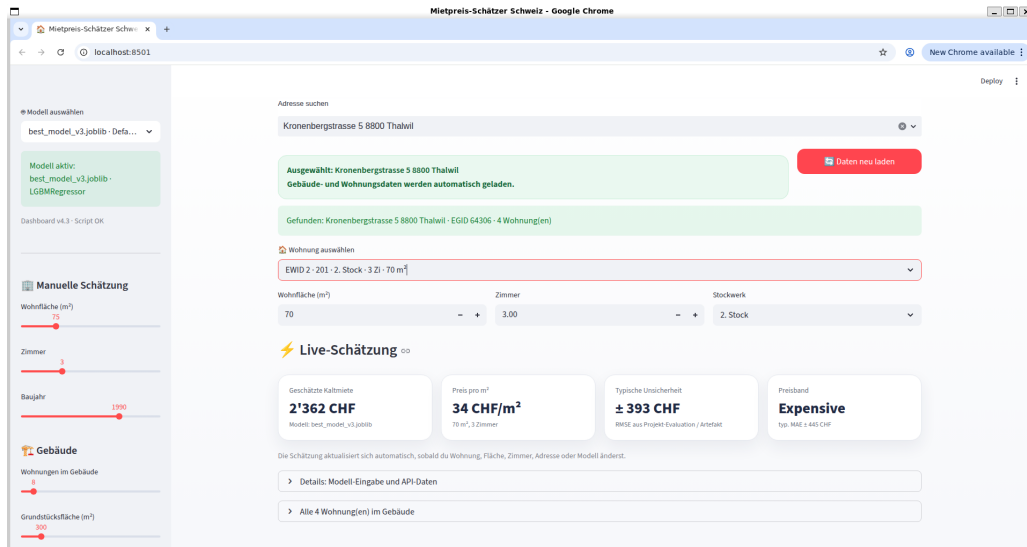


Figure 20: Step 3, Wohnung auswählen. If the GWR record contains multiple dwellings for the building, the user picks the concrete EWID. The Wohnfläche, Zimmerzahl and Stockwerk are then pre-filled from the GWR record. The KPI cards underneath show the live cold-rent estimate (e.g. 2,362 CHF), the price per m² (34 CHF/m²), the typical RMSE-based uncertainty band (±393 CHF) and the price-band classification (“Expensive”).

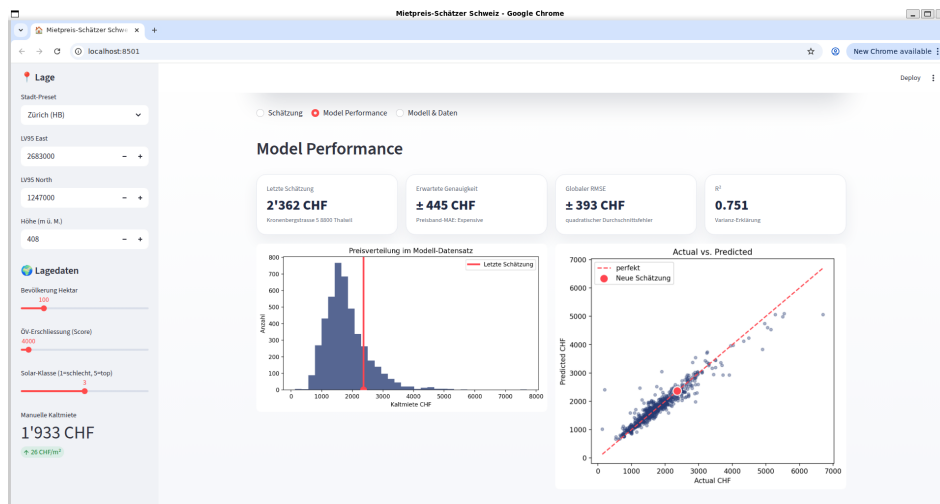


Figure 21: Step 4, Auswertung. The Model-Performance tab compares the current estimate against the training set. The left histogram shows the distribution of cold rents in the modelling set; the red vertical line marks the current estimate. The right panel overlays the current estimate on the Actual-vs-Predicted scatter from Section 5. Together they tell the user how typical (or atypical) the apartment is.

Manuelle Schätzung

Wohnfläche (m²)
75

Zimmer
3

Baujahr
1990

Gebäude

Wohnungen im Gebäude
8

Grundstücksfläche (m²)
300

Lage

Stadt-Preset
Zürich (HB)

LV95 East
2683000

LV95 North
1247000

Höhe (m ü. M.)
408

Lagedaten

Bevölkerung Hektar
100

ÖV-Erschliessung (Score)
4000

Solar-Klasse (1=schlecht, 5=top)
3

Manuelle Kaltmiete
1'933 CHF

↑ 26 CHF/m²

Figure 22: Manuelle Eingabe (fallback). Left: structural inputs — Wohnfläche, Zimmerzahl, Baujahr, Wohnungen im Gebäude, Grundstücksfläche. Right: location inputs — Stadt-Preset (Zürich, Bern, Basel, Genf, ...), LV95 East/North, Höhe, Bevölkerung im Hektar, ÖV-Erschliessungs-Score, Solar-Klasse. Switching the Stadt-Preset pre-fills the coordinates and the population/ÖV defaults, so a user can quickly explore “what would this apartment cost in another city?”. The live estimate updates instantly on every slider change.

6 Conclusions

Summary. The project produced a reproducible regression pipeline for Swiss apartment cold rents. It starts at the custom `rentumo.ch` scraper, runs through enrichment with GWR and swisstopo data, and ends with feature engineering, model training and a structured evaluation. The strongest model in the comparison is a LightGBM regressor that uses the KNN-distance features. On the evaluation split it reaches an RMSE of 393 CHF and an R^2 of 0.751, which

improves on the mid-term mean-prediction baseline of 847 CHF by more than half. The original hypothesis, that a tree-based model outperforms linear regression on this dataset, is therefore clearly supported.

Limitations.

- **Snapshot dataset.** Only one extraction date (13.04.2026). The model cannot react to market trends.
- **Coverage bias.** Urban areas are over-represented; small-town and rural listings are under-represented.
- **Expensive-apartment error.** The model is roughly twice as inaccurate for the top-quartile apartments. The available structural features do not capture luxury attributes (view, finish, floor).
- **Single platform.** All listings come from rentumo.ch. Comparis was explicitly excluded after written notice.
- **No deployment.** The Streamlit demo runs locally only; production hosting is out of scope for DSPRO1.

Future work. A few concrete next steps would have the highest impact:

1. Extract text-based features from the listing descriptions. Attributes such as balcony, parking spot, renovation year, floor level or lake view are likely the missing piece for the expensive apartments, since the current tabular fields do not capture them.
2. Pull listings from a second rental platform, for example homegate.ch or immoscout24.ch. This would reduce the urban coverage bias and roughly double the training set.
3. Replace single-point predictions with prediction intervals using conformal prediction (for example via MAPIE). The demo app could then display estimates as “CHF $X \pm Y$ ” instead of a single number.
4. Move the demo from a local Streamlit instance to a small hosted application on Streamlit Cloud or Render. This is a natural first deliverable for DSPRO2.

7 References

References

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

- [2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [3] Federal Office for Spatial Development (ARE). Public transport accessibility (erreichbarkeit mit dem öffentlichen verkehr), 2025. URL <https://www.are.admin.ch/are/de/home/mobilitaet/grundlagen-und-daten/erreichbarkeit.html>.
- [4] Federal Office of Topography swisstopo. Geoadmin rest api (map.geo.admin.ch), 2025. URL <https://api3.geo.admin.ch/services/sdiservices.html>.
- [5] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.
- [6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.
- [7] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [8] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [9] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.
- [10] Rentumo AG. Rentumo: Swiss rental aggregator, 2026. URL <https://www.rentumo.ch>.
- [11] Sherwin Rosen. Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1):34–55, 1974. doi: 10.1086/260169.
- [12] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022. doi: 10.1016/j.inffus.2021.11.011.
- [13] Swiss Federal Office of Energy (BFE). Solar suitability of roofs (solarenergie-eignung der dächer), 2025. URL <https://www.uvek-gis.admin.ch/BFE/sonnendach/>.
- [14] Swiss Federal Statistical Office (BFS). Federal register of buildings and dwellings (eidgenössisches gebäude- und wohnungsregister, gwr), 2025. URL <https://www.bfs.admin.ch/bfs/de/home/register/gebaeude-wohnungsregister.html>.
- [15] Swiss Federal Statistical Office (BFS). Population statistics on a hectare grid (bevölkerungsstatistik im hektarraster), 2025. URL <https://www.bfs.admin.ch/bfs/de/home/dienstleistungen/geostat.html>.